Prof. Luis Alonso Rivera Oviedo
Instituto Nacional del Puerto de La Libertad

APLICACIÓN LLEGADAS TARDES USANDO INGENIERÍA DE PROMPT

Formato utilizado.

1 **ROL**

Define quién debe comportarse como IA: desarrollador Backend, desarrollador Frontend, administrador de bases de datos, etc.

2 **CONTEXTO**

Describe el proyecto, las tecnologías, la arquitectura y la situación actual.

3 **TAREA**

Explica específicamente qué debe realizar la IA.

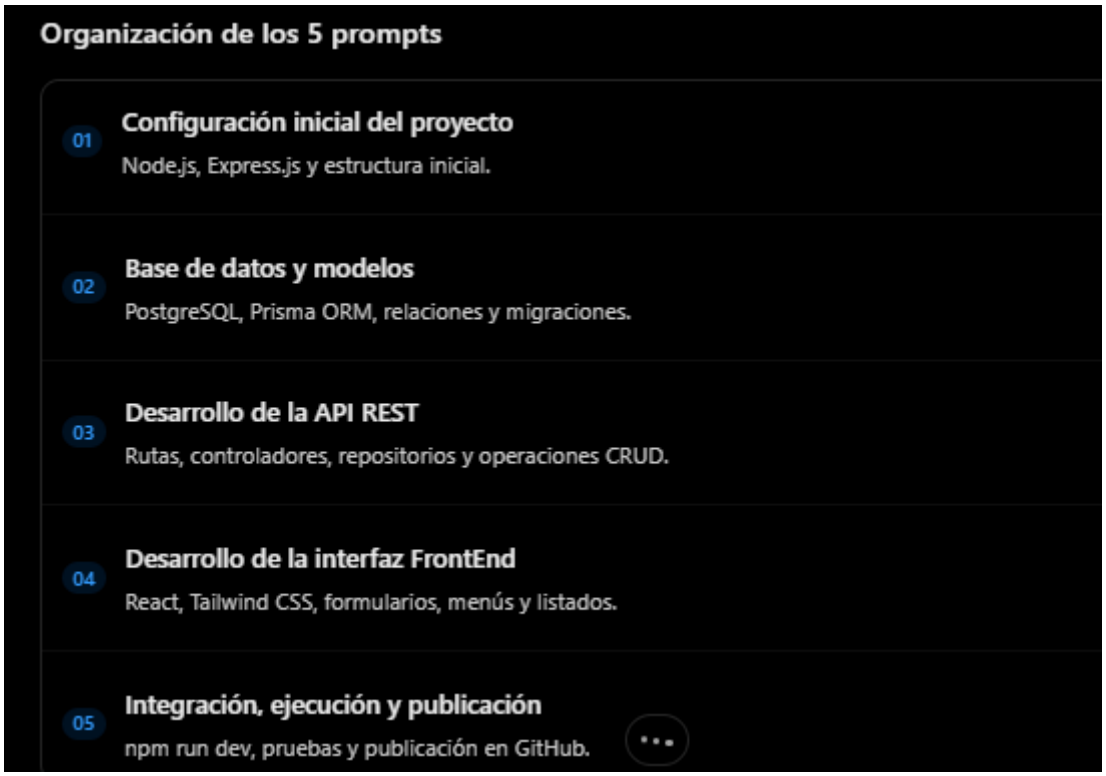
4 **FORMATO**

Establece cómo debe entregar el resultado: archivos, carpetas, código, documentación, etc.

5 **RESTRICCIONES**

Define qué puede y qué no puede hacer la IA, las condiciones que debe respetar y las acciones que requieren autorización.

Para un buen proceso vamos ha organizar los prompt en 5 como máximo.



PROMPT 01 – CONFIGURACIÓN INICIAL DEL PROYECTO

1. ROL
Compórtate como un desarrollador de software especializado en JavaScript, Node.js y Express.js. Aplica buenas prácticas de desarrollo, organización y mantenimiento de proyectos.

2. CONTEXTO
Vamos a desarrollar una aplicación web para gestionar las llegadas tardías de los estudiantes de una institución educativa denominada **Colegio La Pulida S.A.**
El proyecto utilizará las siguientes tecnologías:

- JavaScript.
- Node.js.
- Express.js.
- PostgreSQL como sistema gestor de base de datos.
- Prisma ORM como herramienta de acceso a datos (se configurará en una etapa posterior).
- React para el FrontEnd (se desarrollará posteriormente).

Actualmente nos encontramos en la etapa inicial del proyecto. Trabajaremos directamente en la carpeta en la que se encuentra abierto Visual Studio Code.

3. TAREA

Realiza las siguientes actividades:

1. Examina el contenido de la carpeta actual antes de realizar cualquier modificación.
2. Inicializa un proyecto Node.js si todavía no existe.
3. Configura una API básica utilizando Express.js.
4. Crea el archivo principal del servidor.
5. Configura un puerto para ejecutar el servidor.
6. Implementa una ruta inicial para comprobar que la API funciona correctamente.
7. Organiza el código utilizando una estructura inicial que permita incorporar posteriormente las carpetas de rutas, controladores y repositorios.
8. Configura los scripts necesarios para iniciar el servidor.

4. FORMATO DE ENTREGA

La estructura inicial debe ser clara y organizada. Como mínimo, considera:

- package.json
- src/
- src/server.js
- src/routes/
- .gitignore
- .env.example

Al finalizar, muestra:

- Estructura de carpetas creada.
- Archivos generados.
- Dependencias instaladas.
- Comando para ejecutar la API.
- Dirección local para comprobar su funcionamiento.

5. RESTRICCIONES

- No instales, PostgreSQL, porque ya está instalado en el equipo.
- No configures todavía Prisma ORM.
- No crees modelos ni tablas de la base de datos.
- No desarrolles el FrontEnd.
- No implementes funcionalidades que no hayan sido solicitadas.
- No elimines archivos existentes sin autorización.
- Antes de realizar modificaciones importantes, revisa el contenido actual.
- Cuando termines, detén el trabajo y espera nuevas instrucciones. No avances a la siguiente etapa por iniciativa propia.

---------- fin del prompt1.

PROMPT 02 – CONFIGURACIÓN DE BASE DE DATOS Y MODELOS

1. ROL

Compórtate como un desarrollador Backend especializado en Node.js, PostgreSQL y Prisma ORM. Aplica buenas prácticas de modelado de bases de datos y gestión de relaciones.

2. CONTEXTO

Estamos desarrollando la aplicación **Colegio La Pulida S.A.**, destinada al registro y control de las llegadas tardías de los estudiantes.
El proyecto ya cuenta con una configuración inicial de Node.js y Express.js.
Utilizaremos:

- JavaScript.
- Node.js.
- Express.js.
- PostgreSQL.
- Prisma ORM.

La base de datos se denominará **LlegadasTardes**.
La base de datos PostgreSQL ya se encuentra instalada en el equipo.

3. TAREA

Configura Prisma ORM y desarrolla los modelos de datos de acuerdo con los siguientes requerimientos:

Modelo 1. Estudiantes
Debe contener los siguientes campos:

- ID_Estudiente: identificador principal.
- NIE: número de identificación del estudiante.
- Nombre: nombre completo.
- Responsable: nombre del responsable.
- Telefono: teléfono del responsable.

Modelo 2. Grados
Debe contener:

- ID_Grado: identificador principal.
- Nombre: nombre del grado académico.

Modelo 3. Llegadas Tardes
Debe contener:

- Id_Llegadas Tardes: identificador principal.
- ID_Estudiente: referencia al estudiante.
- ID_Grado: referencia al grado.
- Hora de llegada tardía.
- Fecha.
- Detalle.

Relaciones

Establece las relaciones correspondientes:

- Un estudiante puede registrar múltiples llegadas tardías.
- Un grado puede estar relacionado con múltiples registros de llegadas tardías.
- Cada registro de llegada tardía debe pertenecer a un estudiante y a un grado.

Configuración técnica

1. Instala y configura Prisma ORM.
2. Configura el proveedor PostgreSQL.
3. Define la cadena de conexión mediante variables de entorno.
4. Utiliza la base de datos denominada LlegadasTardes.
5. Configura las credenciales de conexión proporcionadas: usuario y contraseña disponibles en el equipo. La contraseña indicada para esta configuración es admin.
6. Crea los modelos en el archivo de esquema de Prisma.
7. Ejecuta la migración inicial para crear las tablas.
8. Genera el cliente de Prisma.
9. Comprueba que la conexión y la migración hayan finalizado correctamente.

4. FORMATO DE ENTREGA

Organiza los archivos siguiendo la estructura de Prisma.

Entrega:

- Archivo de configuración de Prisma.
- Archivo schema.prisma.
- Archivo .env con la cadena de conexión.
- Migración inicial.
- Modelos y relaciones.
- Comandos utilizados.
- Resultado de la migración.
- Explicación breve de las relaciones establecidas.

5. RESTRICCIONES

- No cambies el nombre de la base de datos.
- No utilices otro ORM.
- No instales ni configures otro gestor de base de datos.
- No modifiques la arquitectura inicial de Express sin necesidad.
- No inventes campos adicionales.
- Los nombres de los identificadores pueden adaptarse a la sintaxis válida de Prisma, conservando el significado de los campos originales.
- No publiques credenciales en GitHub.
- No sobrescribas archivos existentes sin revisarlos.
- No desarrolles todavía los endpoints ni el FrontEnd.
- Al finalizar, espera nuevas instrucciones.

PROMPT 03 – DESARROLLO DE LA API REST

1. ROL

Compórtate como un desarrollador Backend experto en JavaScript, Node.js, Express.js, PostgreSQL y Prisma ORM. Aplica principios de separación de responsabilidades y buenas prácticas de arquitectura de software.

2. CONTEXTO

Estamos desarrollando la aplicación **Colegio La Pulida S.A.**
El proyecto ya cuenta con:

- Servidor Node.js y Express.js.
- Base de datos PostgreSQL denominada LlegadasTardes.
- Prisma ORM configurado.
- Modelos de Estudiantes, Grados y Llegadas Tardes.
- Relaciones entre los modelos.
- Migración inicial ejecutada.

La API debe permitir administrar toda la información de la aplicación.

3. TAREA

Desarrolla los endpoints de la API REST para los tres modelos:

1. Estudiantes.
2. Grados.
3. Llegadas Tardes.

Implementa las operaciones CRUD completas:

- GET: consultar registros.
- POST: insertar nuevos registros.
- PUT: modificar registros.
- PATCH: actualizar parcialmente registros.
- DELETE: eliminar registros.

Arquitectura obligatoria
Organiza el Backend utilizando tres capas principales:
Rutas (Routes)

- Define las direcciones de los endpoints.
- Asocia cada ruta con su controlador correspondiente.

Controladores (Controllers)

- Reciben y procesan las solicitudes HTTP.
- Validan los datos recibidos.
- Gestionan las respuestas.
- Manejan los errores.

Repositorios (Repositories)

- Gestionan las operaciones de acceso a datos.
- Utilizan Prisma ORM para interactuar con PostgreSQL.
- Separan las consultas de la lógica de los controladores.

Organización del proyecto

Toda la API debe organizarse dentro de la carpeta src.
Utiliza una estructura similar a la siguiente:

- src/
    - controllers/
    - repositories/
    - routes/
    - config/
    - server.js

Adapta la estructura si resulta necesario, sin perder la separación de responsabilidades.

Configuración final

Configura el archivo principal de Express para:

- Registrar todas las rutas.
- Configurar el middleware necesario.
- Habilitar CORS para permitir posteriormente la comunicación con React.
- Configurar el procesamiento de JSON.
- Gestionar errores.
- Mantener la conexión con Prisma.

4. FORMATO DE ENTREGA

Entrega:

- Todos los archivos de rutas.
- Todos los controladores.
- Todos los repositorios.
- Configuración principal de Express.
- Lista de endpoints disponibles.
- Métodos HTTP utilizados.
- Ejemplos de solicitudes y respuestas JSON.
- Instrucciones para probar la API mediante una herramienta como Postman.

5. RESTRICCIONES

- Utiliza exclusivamente Prisma ORM para acceder a la base de datos.
- No escribas consultas SQL innecesarias.
- No coloques consultas de base de datos directamente en las rutas.
- Respeta las relaciones establecidas en los modelos.
- Implementa validaciones y manejo de errores.
- No modifiques los modelos ni las migraciones sin autorización.
- No desarrolles todavía la interfaz React.
- No elimines funcionalidades existentes.
- Al finalizar, verifica que los endpoints funcionen y espera nuevas instrucciones.

PROMPT 04 – DESARROLLO DEL FRONTEND CON REACT

1. ROL

Compórtate como un desarrollador FrontEnd especializado en JavaScript, React.js, Tailwind CSS y consumo de API REST. Diseña interfaces modernas, intuitivas, responsivas y fáciles de utilizar.

2. CONTEXTO

Estamos desarrollando la aplicación **Colegio La Pulida S.A.**, destinada a gestionar las llegadas tardías de los estudiantes.
El Backend ya está desarrollado utilizando:

- Node.js.
- Express.js.
- Prisma ORM.
- PostgreSQL.

La API dispone de operaciones CRUD para:

- Estudiantes.
- Grados.
- Llegadas Tardes.

Ahora necesitamos construir una interfaz gráfica que permita utilizar todas las funcionalidades de la API.
El FrontEnd debe desarrollarse dentro de una carpeta denominada **FrontEnd**, ubicada en la raíz del proyecto.
Tecnologías obligatorias:

- JavaScript.
- React.js.
- Tailwind CSS.

3. TAREA

A. Crear el proyecto

1. Crear la carpeta FrontEnd en la raíz.
2. Inicializar el proyecto React.
3. Configurar Tailwind CSS.
4. Organizar los componentes y páginas.
5. Configurar la comunicación con la API mediante solicitudes HTTP.

B. Diseñar la pantalla principal

Crear una pantalla de inicio que incluya:

- Nombre del colegio.
- Menú principal de navegación.
- Diseño visual moderno.
- Imagen de fondo relacionada con un colegio.
- Acceso a las diferentes funcionalidades.

C. Crear formularios de administración

Desarrollar un formulario independiente para cada modelo.

Formulario de Estudiantes

Campos:

- NIE.
- Nombre.
- Responsable.
- Teléfono.

Botones:

- Agregar.
- Guardar.
- Borrar.
- Modificar.
- Eliminar.

Formulario de Grados

Campos:

- Nombre del grado.

Botones:

- Agregar.
- Guardar.
- Borrar.
- Modificar.
- Eliminar.

Formulario de Llegadas Tardes

Campos:

- Estudiante.
- Grado.
- Hora de llegada tardía.
- Fecha.
- Detalle.

Los campos de estudiante y grado deben permitir seleccionar registros existentes.

Botones:

- Agregar.
- Guardar.
- Borrar.
- Modificar.
- Eliminar.

D. Crear listados

Incorporar tres opciones adicionales en el menú principal para consultar:

1. Lista de estudiantes.
2. Lista de grados.
3. Lista de llegadas tardías.

Los listados deben obtener los datos directamente desde la API y mostrar la información de manera organizada.

E. Integración

Todos los formularios y listados deben comunicarse con los endpoints correspondientes.

Implementar:

- Validaciones de campos.
- Mensajes de confirmación.
- Mensajes de error.
- Actualización de listados después de guardar, modificar o eliminar.
- Manejo de estados de carga.

4. FORMATO DE ENTREGA

Organiza el FrontEnd con una estructura clara, por ejemplo:

- FrontEnd/
    - src/
        - components/
        - pages/
        - services/
        - assets/
        - App.jsx
        - main.jsx
    - public/
    - package.json

Entrega todos los componentes y páginas necesarios.

La interfaz debe incluir:

- Menú de navegación.
- Pantalla principal.
- Tres formularios.
- Tres listados.
- Servicios para consumir la API.
- Estilos con Tailwind CSS.

5. RESTRICCIONES

- Utiliza JavaScript, no TypeScript.
- Utiliza React.js.
- Utiliza Tailwind CSS.
- No utilices otro framework FrontEnd.
- No modifiques el Backend sin autorización.
- No alteres los modelos de la base de datos.
- No utilices datos ficticios cuando la información debe proceder de la API.
- No dejes botones sin funcionalidad.
- Mantén una interfaz responsiva.
- Respeta la estructura de carpetas existente.
- Al finalizar, comprueba la comunicación con la API y espera nuevas instrucciones.

PROMPT 05 – INTEGRACIÓN, EJECUCIÓN Y PUBLICACIÓN DEL PROYECTO

1. ROL

Compórtate como un desarrollador Full Stack especializado en Node.js, Express.js, React.js, Prisma ORM, PostgreSQL, npm y Git/GitHub.
Tu responsabilidad es integrar los componentes desarrollados, verificar su funcionamiento y preparar la publicación del proyecto siguiendo buenas prácticas.

2. CONTEXTO

Hemos desarrollado la aplicación **Colegio La Pulida S.A.**, organizada en dos componentes principales:

Backend:

- Node.js.
- Express.js.
- Prisma ORM.
- PostgreSQL.
- API REST.
- Arquitectura de rutas, controladores y repositorios.

Frontend:

- React.js.
- JavaScript.
- Tailwind CSS.
- Formularios y listados.
- Consumo de la API.

La estructura general del proyecto es:

- APPTardes/
    - src/
    - FrontEnd/
    - package.json
    - .gitignore

Actualmente necesitamos configurar la ejecución integrada y publicar todo el proyecto en GitHub.

3. TAREA

A. Configurar ejecución integrada

Configura el proyecto para que tanto el Backend como el FrontEnd puedan ejecutarse desde la raíz utilizando un único comando:

npm run dev

Para conseguirlo:

1. Revisa los archivos package.json existentes.
2. Configura los scripts necesarios.
3. Utiliza una herramienta adecuada para ejecutar simultáneamente ambos servicios.
4. Mantén los comandos individuales para ejecutar cada componente por separado.
5. Comprueba que ambos servidores inicien correctamente.
6. Verifica que React pueda comunicarse con la API.

B. Verificar el funcionamiento

Realiza pruebas para comprobar:

- Inicio del Backend.
- Inicio del FrontEnd.
- Conexión con PostgreSQL.
- Funcionamiento de Prisma.
- Comunicación entre React y Express.
- Registro de estudiantes.
- Registro de grados.
- Registro de llegadas tardías.
- Modificación y eliminación de registros.
- Visualización de los listados.

Corrige los errores encontrados sin modificar funcionalidades que ya funcionan correctamente.

C. Preparar el proyecto para GitHub

Configura o revisa el archivo .gitignore.

Excluye archivos y carpetas innecesarios, especialmente:

- node_modules/
- Archivos .env.
- Archivos de compilación.
- Archivos temporales.
- Logs.
- Archivos generados que no deban versionarse.

Conserva los archivos necesarios para instalar las dependencias y reproducir las migraciones.

D. Publicar en GitHub

El repositorio de destino es:

https://github.com/profeluisindel/Aplicacion_LLegadasTardes

Realiza las siguientes actividades:

1. Verifica si el proyecto ya tiene inicializado Git.
2. Comprueba la configuración del repositorio remoto.
3. Verifica el estado de los archivos.
4. Prepara los archivos que deben incluirse.
5. Realiza el commit correspondiente.
6. Publica el proyecto en el repositorio indicado.

4. FORMATO DE ENTREGA

Al finalizar, proporciona:

- Estructura definitiva del proyecto.
- Scripts configurados en package.json.
- Comando para iniciar toda la aplicación.
- Resultado de las pruebas realizadas.
- Archivos excluidos mediante .gitignore.
- Dirección del repositorio GitHub.
- Resumen de los cambios publicados.

5. RESTRICCIONES

- No modifiques funcionalidades que ya funcionan correctamente.
- No cambies las tecnologías utilizadas.
- No elimines archivos importantes.
- No publiques archivos .env ni contraseñas.
- No subas carpetas node_modules.
- No sobrescribas un repositorio remoto sin comprobar previamente su contenido.
- No incluyas archivos innecesarios.
- Si existen conflictos con GitHub, informa antes de resolverlos mediante operaciones destructivas.
- No inventes resultados de pruebas ni confirmes una publicación que no se haya realizado.
- Solicita autorización antes de realizar operaciones que puedan sobrescribir o eliminar información del repositorio remoto.

Fin de los prompt.